



Van bit tot bug: *Doorheen de lagen van de systeemstack*

Jo Van Bulck

🏠 DistriNet, KU Leuven, Belgium ✉ jo.vanbulck@kuleuven.be

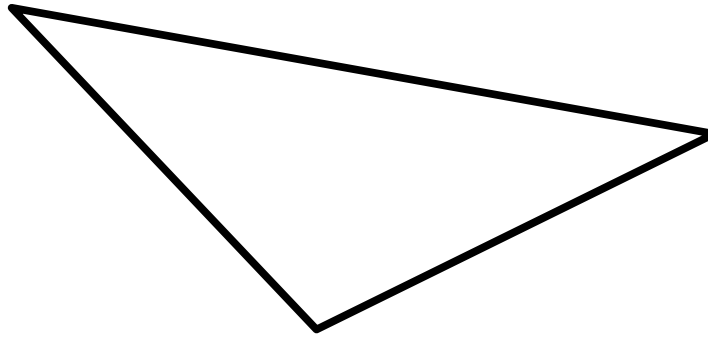
🌐 <https://vanbulck.net>

KU Leuven Startdagen, 16/09/2026

Welkom aan de Universiteit!



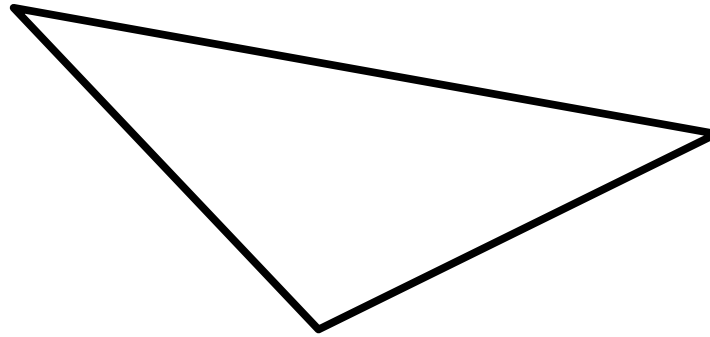
Educatie



Welkom aan de Universiteit!



Educatie



Onderzoek



Maatschappelijke dienstverlening



Onderzoek: ik breek computers
om ze veiliger te maken

- Onderzoeksgroep **DistriNet**
- HW/SW interface
 - **Microarchitectural** attacks and defenses
 - **Trusted execution** environments



Onderwijs: waar kom je me tegen?



- **Besturingsystemen** (2e ba)
- **Wetenschapscommunicatie** (2e ba)
- Wetenschappelijke vorming (3e ba)
- Development of Secure Software (ma)



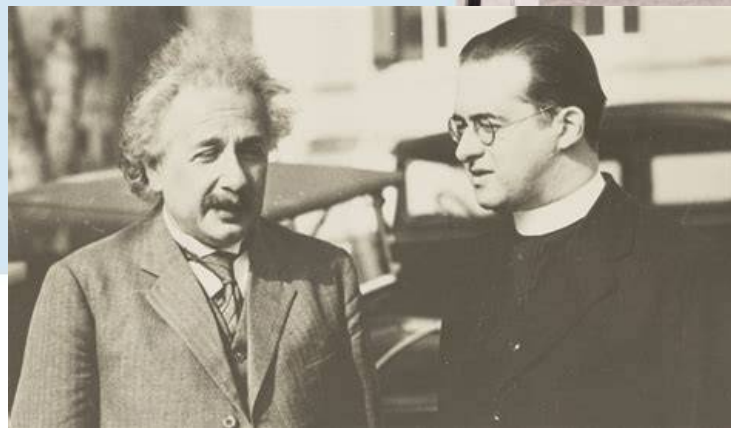
Faculteit Wetenschappen KU Leuven

[Onderzoek](#) [Studeren](#) [Outreach](#) [International](#) [Fonds](#) [Over ons](#)

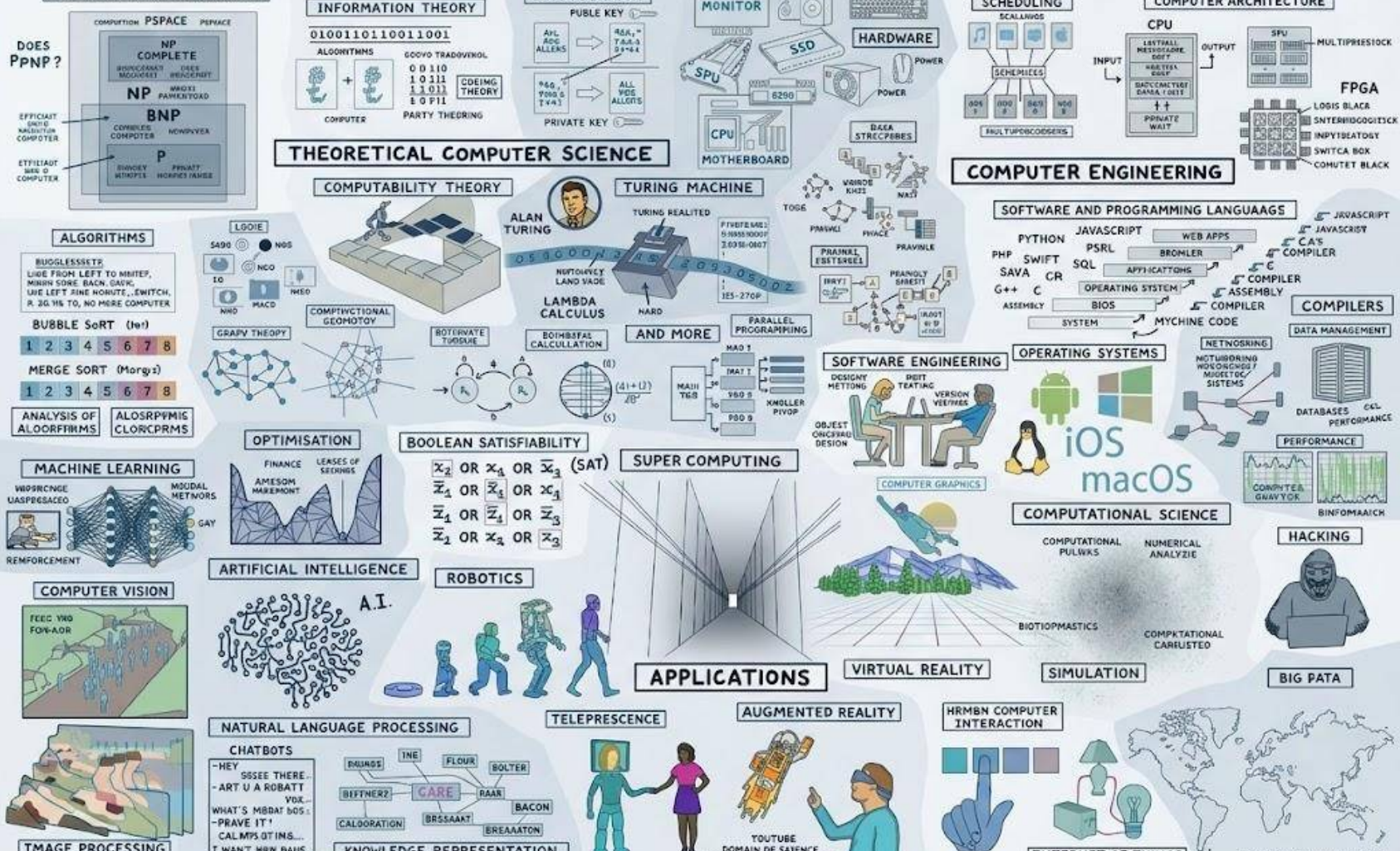
NL 

Wetenschappelijke vooruitgang is het ontdekken van een meer en meer begrijpelijke eenvoud

~ Georges Lemaître



COMPUTATIONAL COMPLEXITY CRYPTOGRAPHY COMPUTER ARCHITECTURE



Begrijpelijke eenvoud: “1e wet van computerwetenschappen”



*“Any problem in Computer Science can be solved with **another layer of indirection**”*

– Butler W. Lampson



Begrijpelijke eenvoud: “1e wet van computerwetenschappen”



*“Any problem in Computer Science can be solved with **another layer of indirection**”*

– Butler W. Lampson

... except for the problem of too many levels of indirection ;-)



De kracht van abstracties: De systeemstack

AN x64 PROCESSOR IS SCREAMING ALONG AT BILLIONS OF CYCLES PER SECOND TO RUN THE XNU KERNEL, WHICH IS FRANTICALLY WORKING THROUGH ALL THE POSIX-SPECIFIED ABSTRACTION TO CREATE THE DARWIN SYSTEM UNDERLYING OS X, WHICH IN TURN IS STRAINING ITSELF TO RUN FIREFOX AND ITS GECKO RENDERER, WHICH CREATES A FLASH OBJECT WHICH RENDERS DOZENS OF VIDEO FRAMES EVERY SECOND

BECAUSE I WANTED TO SEE A CAT JUMP INTO A BOX AND FALL OVER.

I AM A GOD.

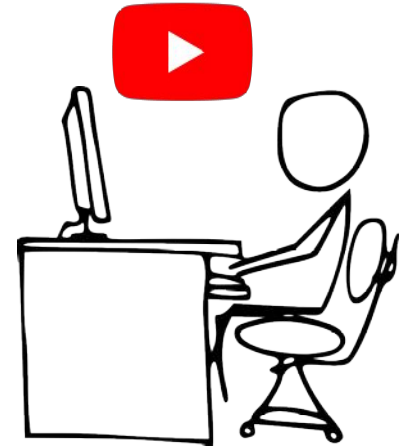
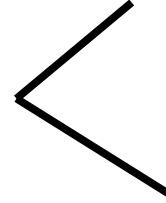
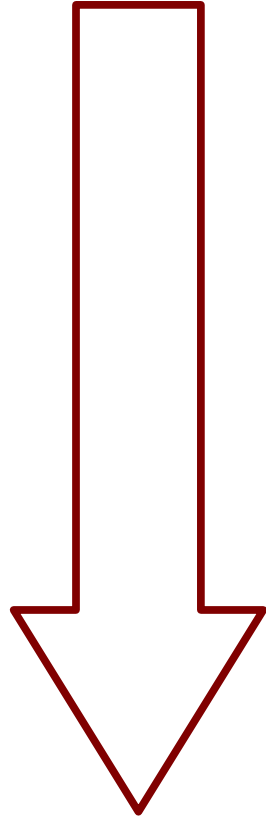


- Toren/**stack** van **abstracties**
- **Interface** verbergt de **complexiteit** van de **implementatie**
- Elke **laag** bouwt verder op **functionaliteit** eronder

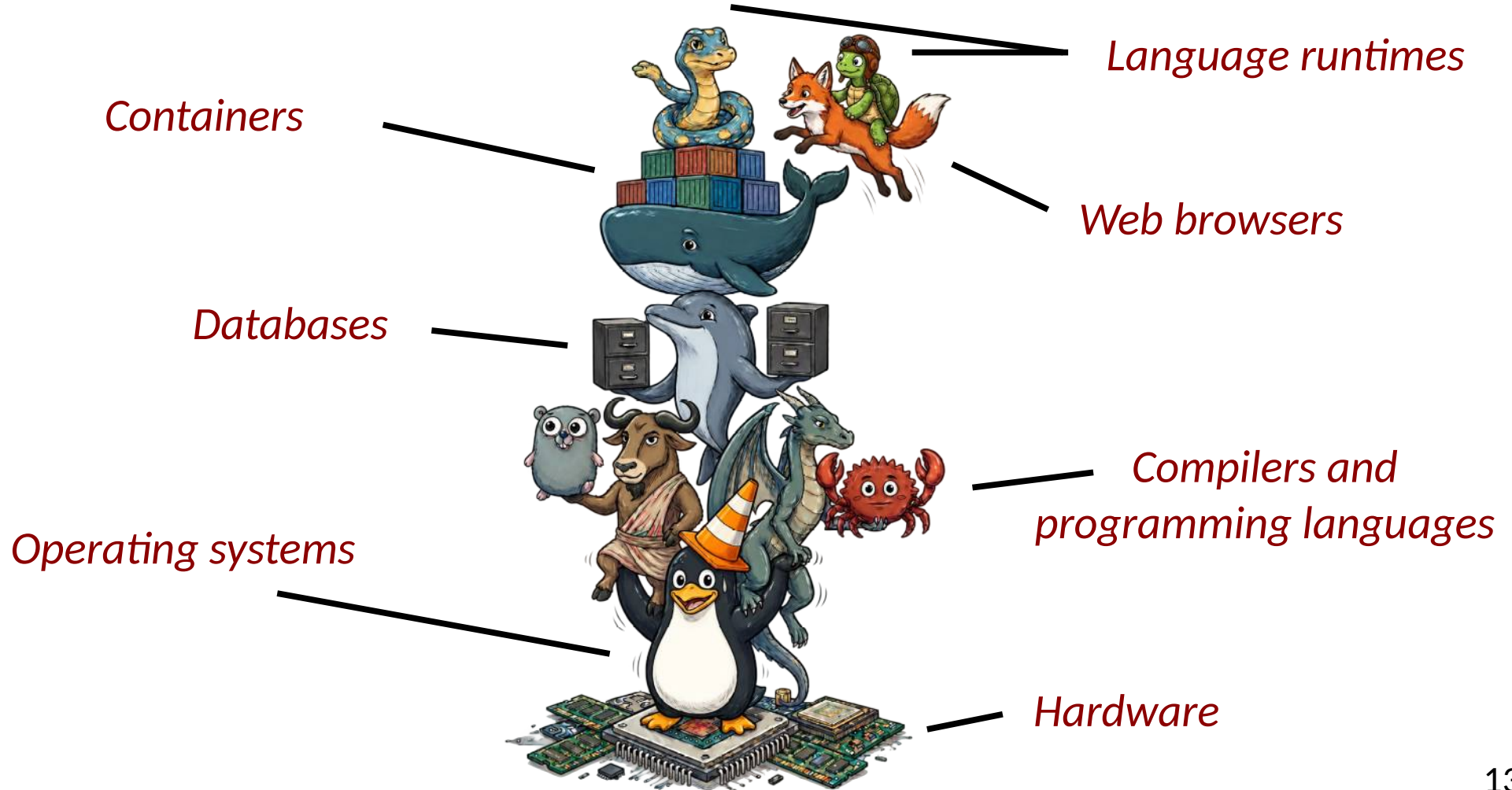
Doorheen de lagen van de systeemstack...



Doorheen de lagen van de systeemstack...

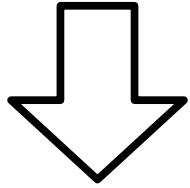


Doorheen de lagen van de systeemstack...



Doorheen de lagen van de systeemstack: Studieprogramma

**Beginnelen van programmeren;
modulair programmeren**
(1e ba)



top-down

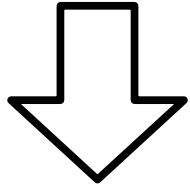
**Comparitive Study of
Programming languages**
(2e ma)

Compilerconstructie
(1e/2e ma)



Doorheen de lagen van de systeemstack: Studieprogramma

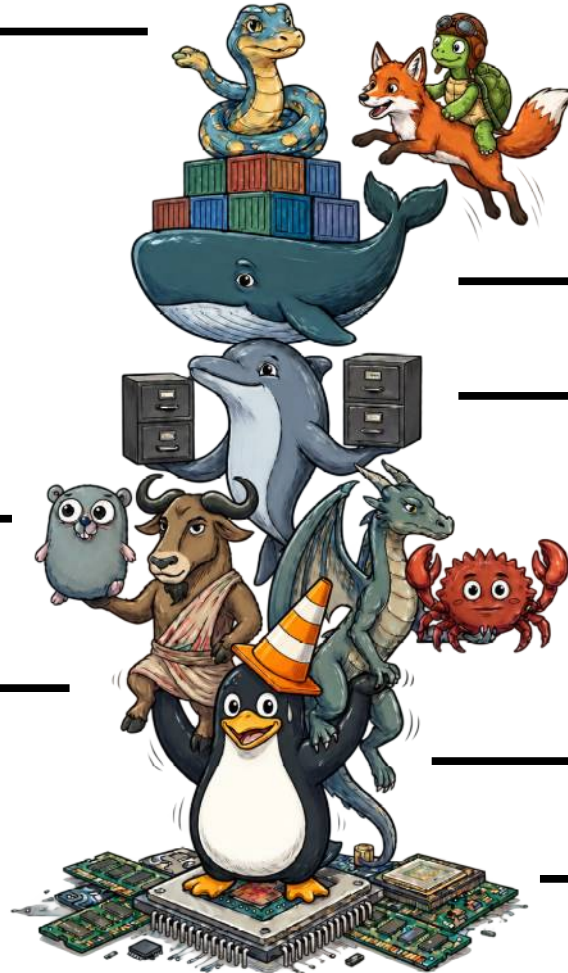
**Beginnelen van programmeren;
modulair programmeren**
(1e ba)



top-down

**Comparitive Study of
Programming languages**
(2e ma)

Compilerconstructie
(1e/2e ma)



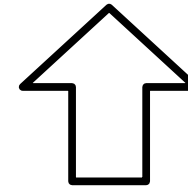
Computer networks (3e ba)

Distributed systems (1e ma)

Gegevensbanken (1e ba)

Besturingssystemen (2e ba)

Computerarchitectuur (2e ba)



bottom-up

Overzicht: Van bit tot bug...

2. Memory-safety attacks
(C → machine code)



1. SQL injection attacks
(mens → programmeertaal)

3. Microarchitectural attacks
(machine code → hardware)

Overzicht: Van bit tot bug...



2. Memory-safety attacks
(C → machine code)

1. SQL injection attacks
(mens → programmeertaal)

3. Microarchitectural attacks
(machine code → hardware)

Gegevensbanken 101: Tabel layout

 wachtwoord	 naam	 email	 grades
pass123	Jo	jo@school.edu	A+
secret88	Anna	anna@school.edu	B
qwerty99	Mark	mark@school.edu	A-
stud2026	Joana	joana@school.edu	B+

Gegevensbanken 101: Structured Query Language (SQL)

```
SELECT * FROM studenten WHERE naam = 'Jo';
```



SELECT *

The asterisk (*) requests **all columns** (wachtwoord, naam, email, grades) for the matching rows.



FROM studenten

Specifies the target table name in the database where the records are stored.



WHERE naam = 'Jo'

Applies a logical condition. Only rows where the field naam is exactly equal to 'Jo' will be returned.

Gegevensbanken 101: Structured Query Language (SQL)

```
SELECT * FROM studenten WHERE naam = 'Jo';
```

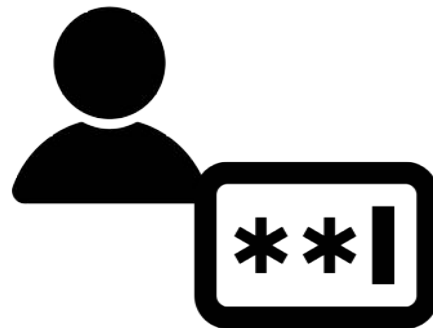
wachtwoord	naam	email	grades
pass123	Jo MATCH	jo@school.edu	A+
secret88	Anna	anna@school.edu	B
qwerty99	Mark	mark@school.edu	A-
stud2026	Joana	joana@school.edu	B+

SQL injectie: Inputinterpretatie over abstractielagen

```
/* naam = wat de gebruiker intypt */  
query = "SELECT * FROM studenten "  
        + "WHERE naam = '" + naam + "'" "  
        + "AND wachtwoord = '" + pwd + "'";
```

Wat je verwacht:

```
SELECT * FROM studenten  
WHERE naam = 'Jo' AND wachtwoord = 'pass123'
```



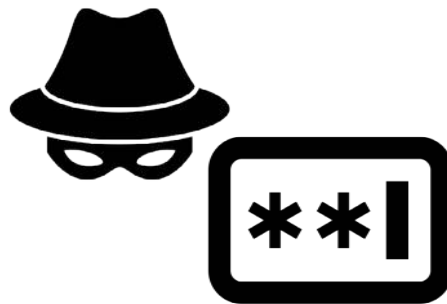
SQL injectie: Inputinterpretatie over abstractielagen

```
/* naam = wat de gebruiker intypt */  
query = "SELECT * FROM studenten "  
        + "WHERE naam = '" + naam + "'" "  
        + "AND wachtwoord = '" + pwd + "'";
```

Wat een aanvaller intypt

```
SELECT * FROM studenten  
WHERE naam = 'Jo' AND wachtwoord = '' OR '1'  
= '1' <- altijd waar!
```

=> de databank geeft ALLE studenten terug



SQL injectie: Inputinterpretatie over abstractielagen

```
/* naam = wat de gebruiker intypt */  
query = "SELECT * FROM studenten "  
        + "WHERE naam = '" + naam + "'" "  
        + "AND wachtwoord = '" + pwd + "'";
```

Wat een aanvaller intypt

```
SELECT * FROM studenten  
WHERE naam = 'Jo' AND wachtwoord = '' OR '1'  
= '1' <- altijd waar!
```

=> de databank geeft ALLE studenten terug

Waarom lukt dit?

De databank krijgt één lange string:

- Programmeur: *trusted input*
- Gebruiker: *untrusted input*

→ Code en data door elkaar(!)

Zelfde bug, andere laag:

- command injection (shell)
- XSS (browser)

Lesson: SQL Injection Public preview

Want everyone on your software team to learn this? Get completion tracking and compliance reporting with Hacksplaining for Teams. Free 14-day trial.

[Train my team →](#)

Go ahead and try logging in with the following credentials: Email: `user@email.com` Password: `password`



← →

Username

Password

Log in

An error occurred

Credentials did not match, login failed.

```
Application initialized.User is attempting to login...
SELECT * FROM users WHERE email = 'user@email.com' AND password = 'password'
Credentials did not match, login failed.
```

Lab progress



3 / 14

[← Back](#)

Stage 3 of 14

[Continue →](#)

HI, THIS IS
YOUR SON'S SCHOOL.
WE'RE HAVING SOME
COMPUTER TROUBLE.



OH, DEAR - DID HE
BREAK SOMETHING?
IN A WAY -



DID YOU REALLY
NAME YOUR SON
Robert'); DROP
TABLE Students;-- ?



OH. YES. LITTLE
BOBBY TABLES,
WE CALL HIM.

WELL, WE'VE LOST THIS
YEAR'S STUDENT RECORDS.
I HOPE YOU'RE HAPPY.



AND I HOPE
YOU'VE LEARNED
TO SANITIZE YOUR
DATABASE INPUTS.

Overzicht: Van bit tot bug...



2. Memory-safety attacks
(C → machine code)

1. SQL injection attacks
(mens → programmeertaal)

3. Microarchitectural attacks
(machine code → hardware)

Illusie: Wat we zien in een programmeertaal (C)

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      gets(name);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```

Illusie: Wat we zien in een programmeertaal (C)

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      gets(name);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```

Toegewezen variabelen:

- isAdmin is ingesteld op 0 (standaard geweigerd)
- name array bevat maximaal 8 tekens

Illusie: Wat we zien in een programmeertaal (C)

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      gets(name);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```

Toegewezen variabelen:

- isAdmin is ingesteld op 0 (standaard geweigerd)
- name array bevat maximaal 8 tekens

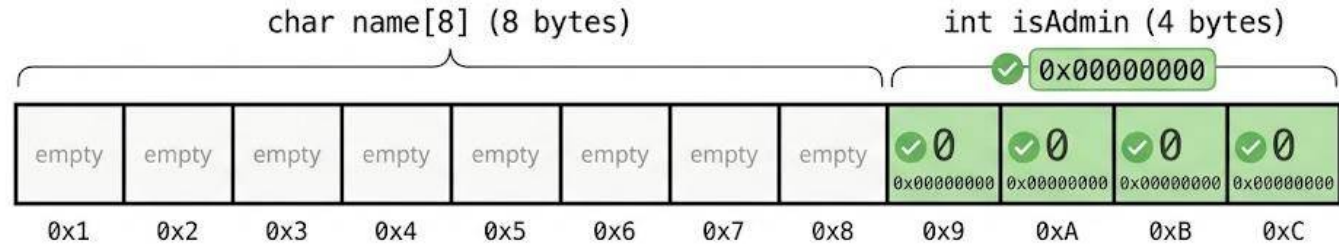


Verwachte Uitvoering:

isAdmin = 0 → access denied

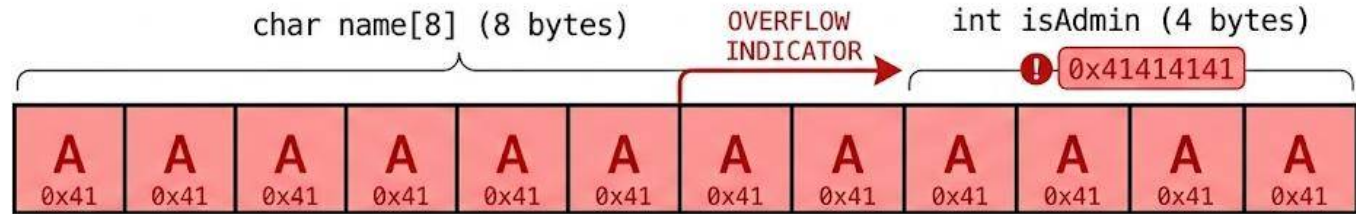
Buffer overflow: onder de abstractie zitten bytes

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      gets(name);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```



Buffer overflow: onder de abstractie zitten bytes

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      gets(name);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```



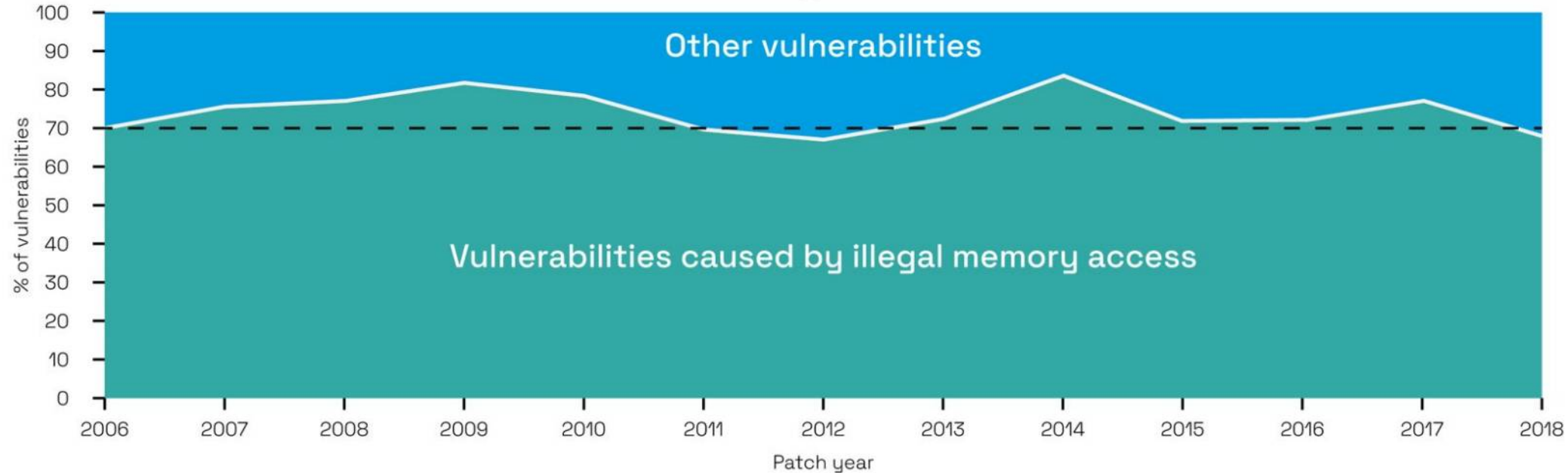
Buffer overflow: onder de abstractie zitten bytes

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int isAdmin = 0;
6      char name[8];
7
8      printf(format: "Enter name: ");
9      fgets(s: name, n: sizeof(name), stream: stdin);
10
11     if (isAdmin != 0) {
12         printf(format: "hello %s: access granted!\n", name);
13     } else {
14         printf(format: "hello %s: access denied!\n", name);
15     }
16     return 0;
17 }
```



Always include bounds checks!

Relevantie van buffer overflows “in the wild”



Source: "Trends, challenges and shifts in vulnerability mitigation", Matt Miller (MSRC), BlueHat IL 2019.

Overzicht: Van bit tot bug...

2. Memory-safety attacks
(C → machine code)

1. SQL injection attacks
(human → programmeertaal)

3. Microarchitectural attacks
(machine code → hardware)



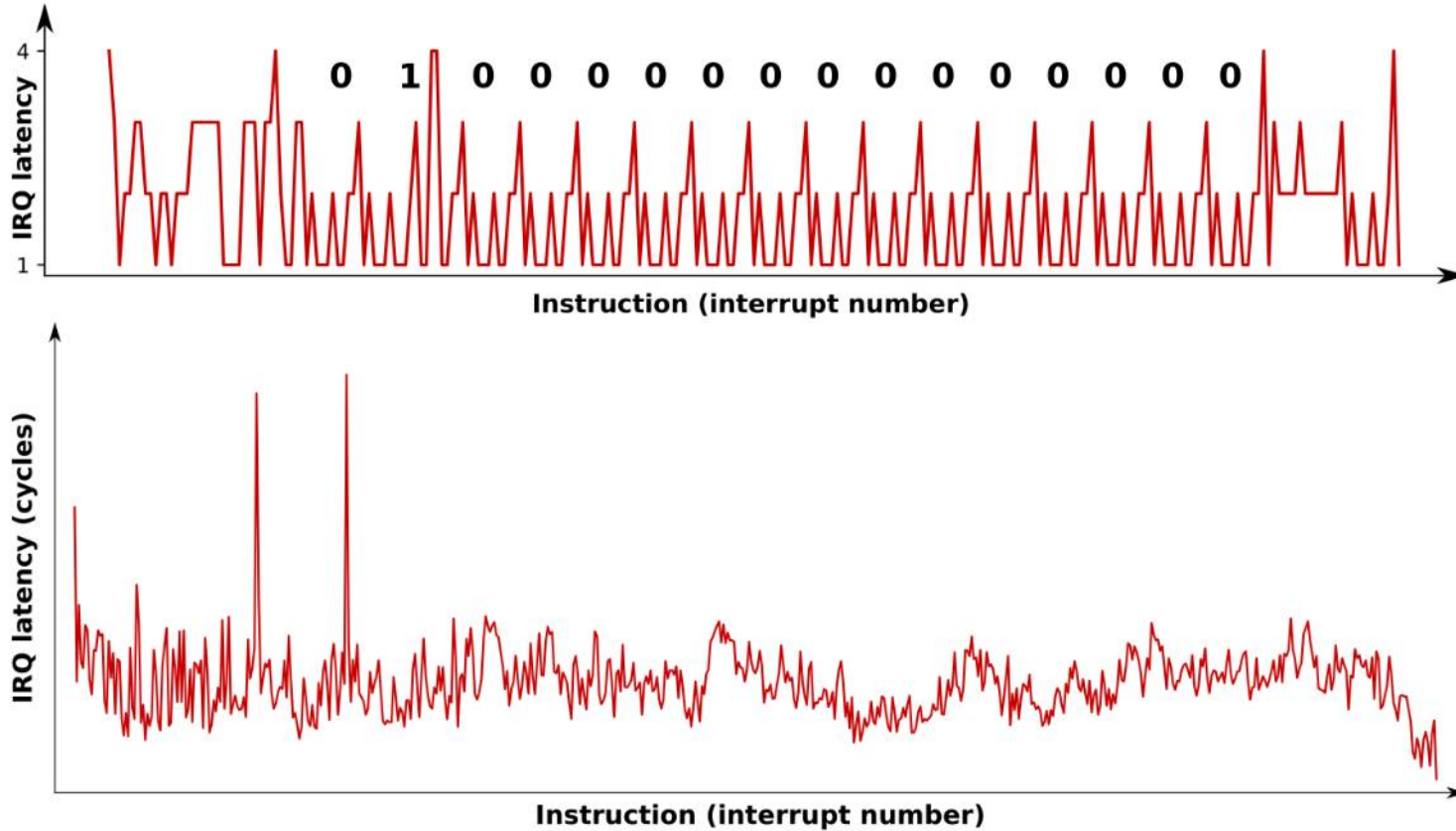


VAULT DOOR

WEIGHT: 22 1/2 Tons
THICKNESS: 22 Inches
STEEL: 11 Layers of Special
Cutting and Drill Resistant
LOCKS: 4 Hamilton Watch
Movements for Time Locks



Microarchitectural Timing Leaks in Practice

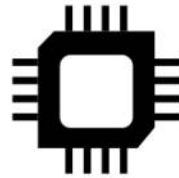


Example: CPU Cache Timing Side Channel



Cache principle: CPU speed $\gg$ DRAM $\rightarrow$ *cache code/data*

```
while true do  
  maccess(&a);  
endwh
```



CPU + cache



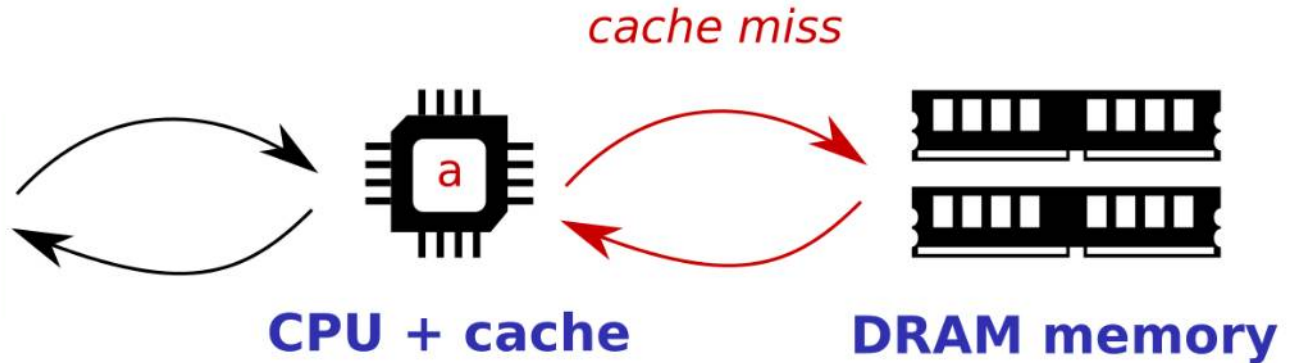
DRAM memory

Example: CPU Cache Timing Side Channel



Cache miss: Request data from (slow) DRAM upon first use

```
while true do  
  maccess(&a);  
endwh
```

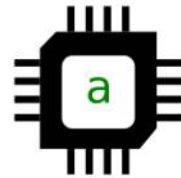
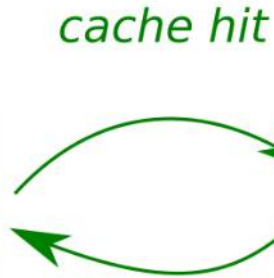


Example: CPU Cache Timing Side Channel



Cache hit: No DRAM access required for subsequent uses

```
while true do  
  maccess(&a);  
endwh
```

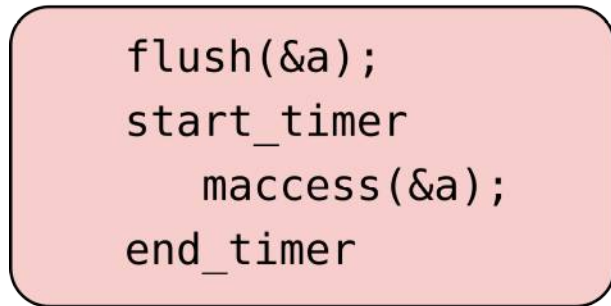


CPU + cache

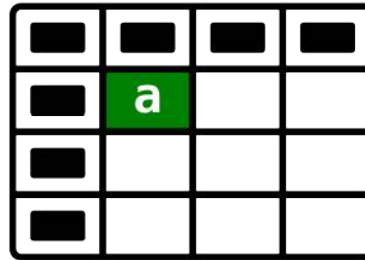


DRAM memory

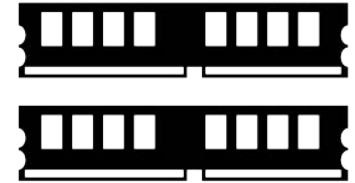
Cache Timing Attacks in Practice: Flush+Reload



*'a' is accessible
to attacker*

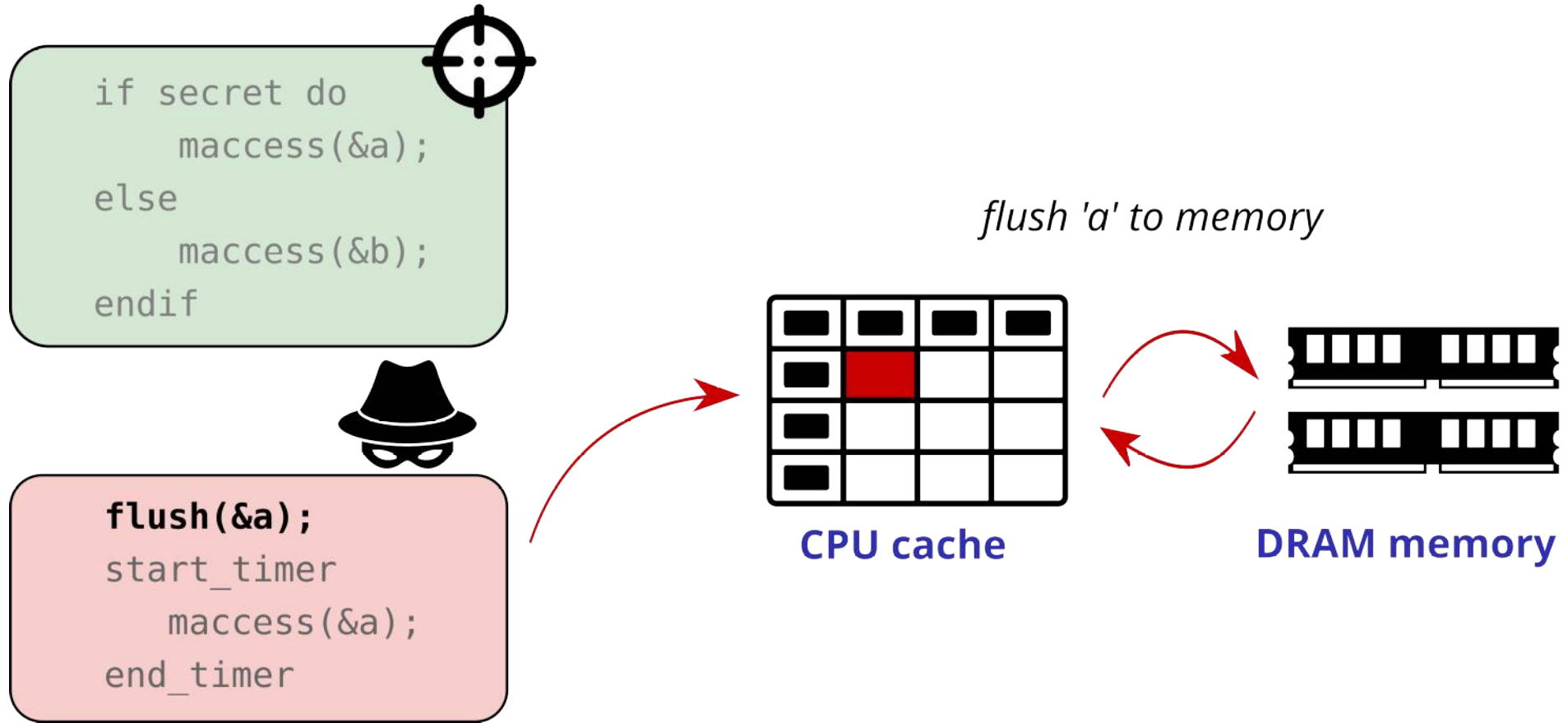


CPU cache

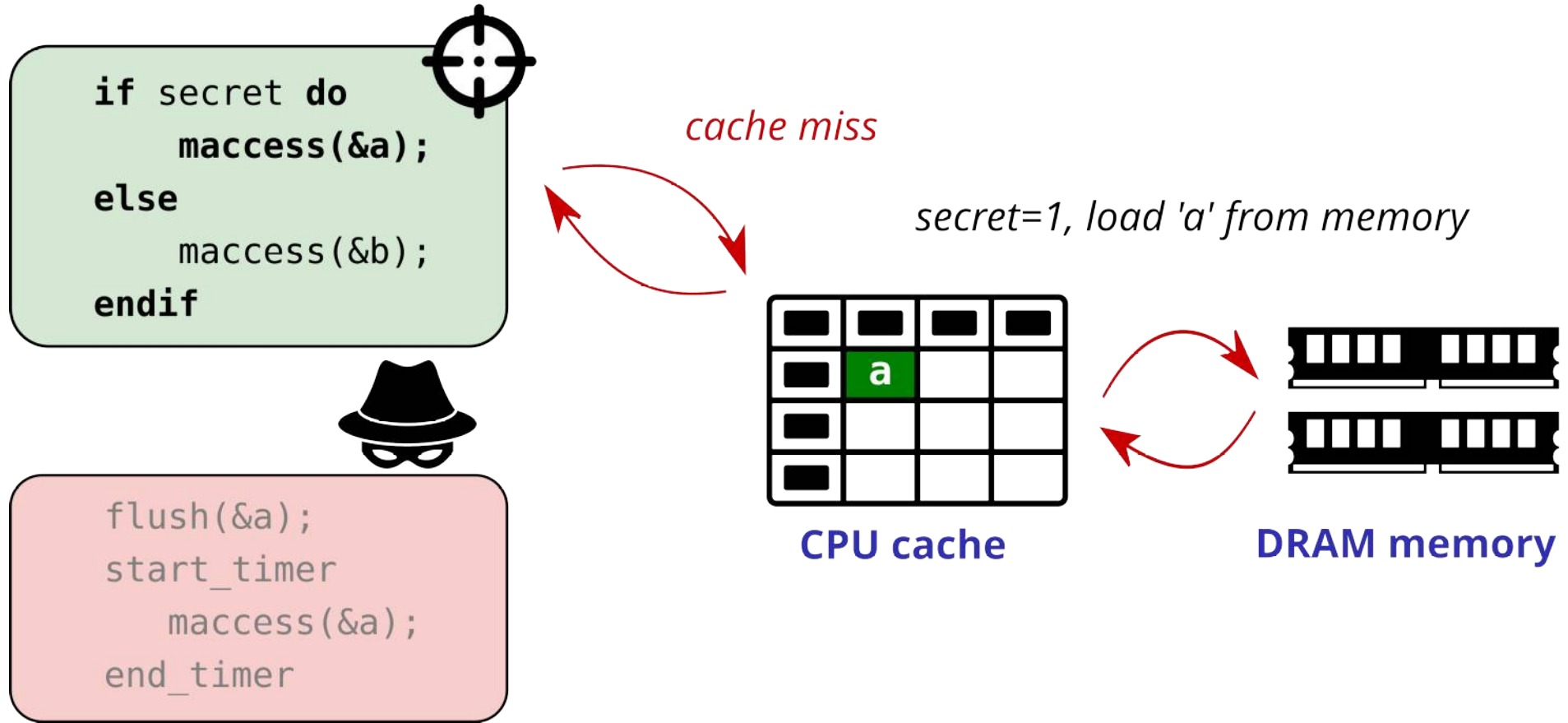


DRAM memory

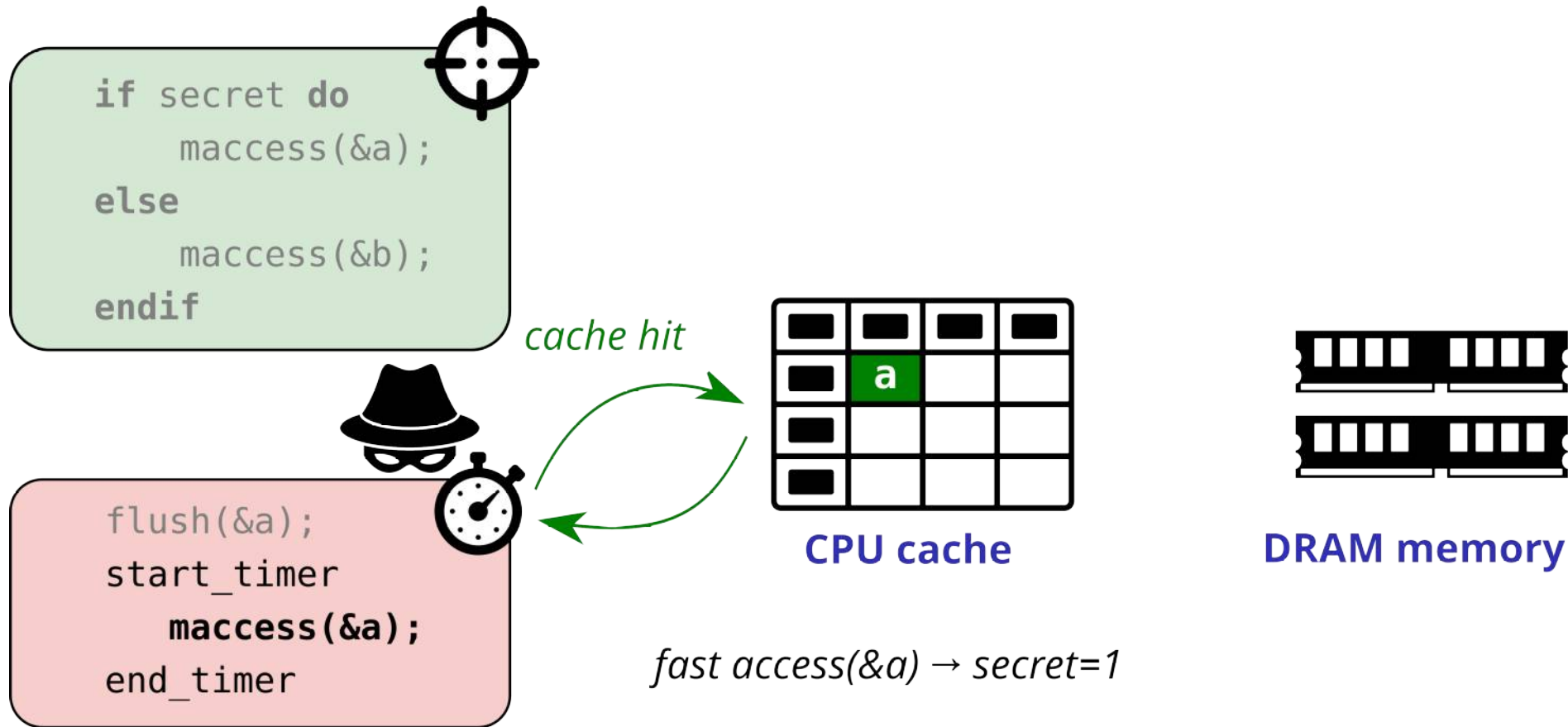
Cache Timing Attacks in Practice: Flush+Reload



Cache Timing Attacks in Practice: Flush+Reload



Cache Timing Attacks in Practice: Flush+Reload



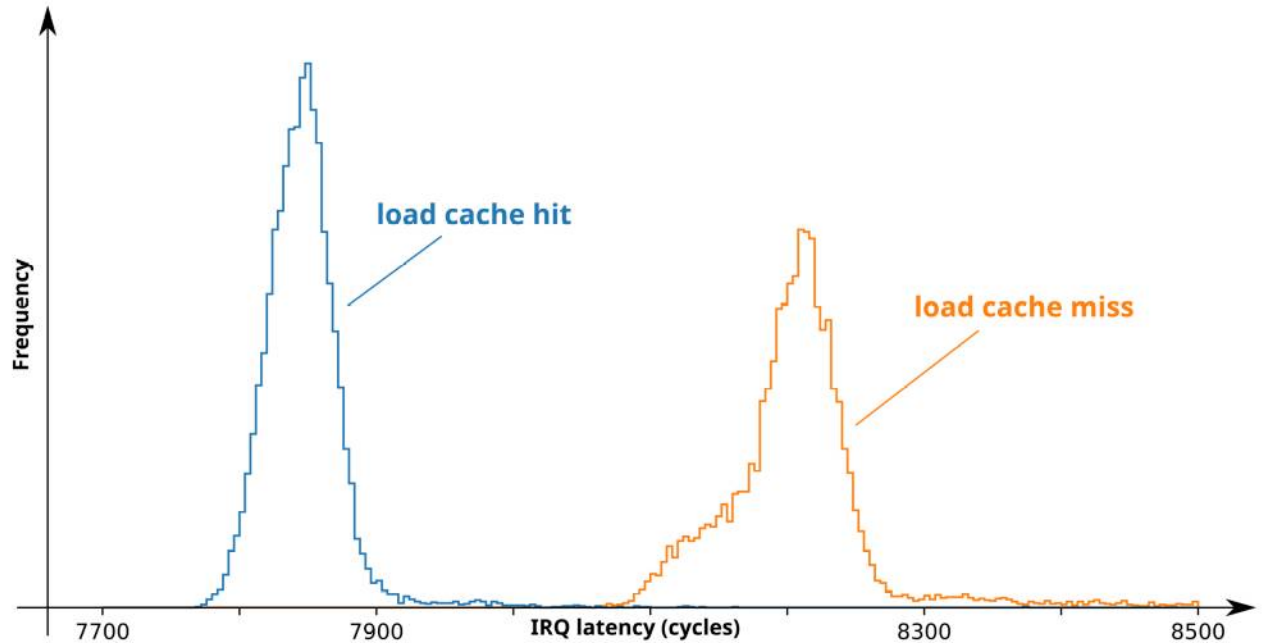
Cache Timing Attacks in Practice: Flush+Reload



```
if secret do
    maccess(&a);
else
    maccess(&b);
endif
```



```
flush(&a);
start_timer
    maccess(&a);
end_timer
```



Demo: Spying on Keystrokes with Flush+Reload

```
25336620182821: Cache Hit (218 cycles) after a pause of 3 cycles
25336620297955: Cache Hit (216 cycles) after a pause of 43 cycles
25336620341217: Cache Hit (216 cycles) after a pause of 15 cycles
25336620363985: Cache Hit (212 cycles) after a pause of 4 cycles
25336620483903: Cache Hit (218 cycles) after a pause of 42 cycles
25336620499835: Cache Hit (216 cycles) after a pause of 3 cycles
25336620552419: Cache Hit (216 cycles) after a pause of 19 cycles
25336621476911: Cache Hit (218 cycles) after a pause of 300 cycles
25336974127733: Cache Hit (220 cycles) after a pause of 104704 cycles
25337739302241: Cache Hit (214 cycles) after a pause of 263629 cycles
25337739686069: Cache Hit (218 cycles) after a pause of 116 cycles
25337739773947: Cache Hit (218 cycles) after a pause of 27 cycles
25337739997613: Cache Hit (228 cycles) after a pause of 84 cycles
25338346337023: Cache Hit (228 cycles) after a pause of 211810 cycles
25338346617849: Cache Hit (224 cycles) after a pause of 81 cycles
25338346627851: Cache Hit (228 cycles) after a pause of 2 cycles
25338346634917: Cache Hit (228 cycles) after a pause of 1 cycles
25338346653587: Cache Hit (222 cycles) after a pause of 5 cycles
25338346811743: Cache Hit (220 cycles) after a pause of 58 cycles
25338346899541: Cache Hit (222 cycles) after a pause of 35 cycles
25338346911083: Cache Hit (222 cycles) after a pause of 3 cycles
25339081895869: Cache Hit (204 cycles) after a pause of 268339 cycles
25339081934737: Cache Hit (228 cycles) after a pause of 3 cycles
25339082052305: Cache Hit (226 cycles) after a pause of 34 cycles
25339082092569: Cache Hit (228 cycles) after a pause of 8 cycles
25339082116253: Cache Hit (224 cycles) after a pause of 3 cycles
25339082273651: Cache Hit (202 cycles) after a pause of 53 cycles
25339815487639: Cache Hit (226 cycles) after a pause of 232157 cycles
```

```
3 super secret keystroke timings
```

```
4
```

```
5 F
```

https://github.com/isec-tugraz/cache_template_attacks

Van bit tot bug: wat onthoud je?

1. Kracht van abstracties

→ Elke laag verbergt de **complexiteit** van de lagen eronder

2. Maar geen enkele abstractie is perfect

→ **Aanvallers** breken door abstractielagen

3. Wie de lagen kent, ziet de bugs

→ Belang van de *top-down* en *bottom-up* exploratie van de **stroomstapel** in de komende jaren :-)

Happy hacking!

Vragen of nieuwsgierig? jo.vanbulck@kuleuven.be

